



# Chapter 4 - Pattern Mining

J. Han, J. Pei and H. Tong, *Data Mining, Concepts and Techniques*, 4<sup>th</sup> ed., Morgan Kaufmann, 2023.



# Contents

- 1. Basic Concepts
- 2. Frequent Itemset Mining Methods
- 3. Which Patterns Are Interesting? Pattern Evaluation Methods



# 1. Basic Concepts

- **Frequent patterns** are patterns (e.g., itemsets, subsequences, or substructures) that appear frequently in a data set.
  - For example, a set of items, such as milk and bread, that appear frequently together in a transaction data set is a *frequent itemset*.
  - A subsequence, such as buying first a smartphone, then a smart TV, and then a smart home device, if it occurs frequently in a shopping history database, is a *(frequent) sequential pattern*.
  - A *substructure* can refer to different structural forms, such as subgraphs, subtrees, or sublattices. If a substructure occurs frequently, it is called a *(frequent) structured pattern*.
- Finding frequent patterns plays an essential role in mining associations, correlations, and many other interesting relationships among data. Moreover, it helps in data classification, clustering, and other data mining tasks.
- Thus, frequent pattern mining has become an important data mining task and a focused theme in data mining research.



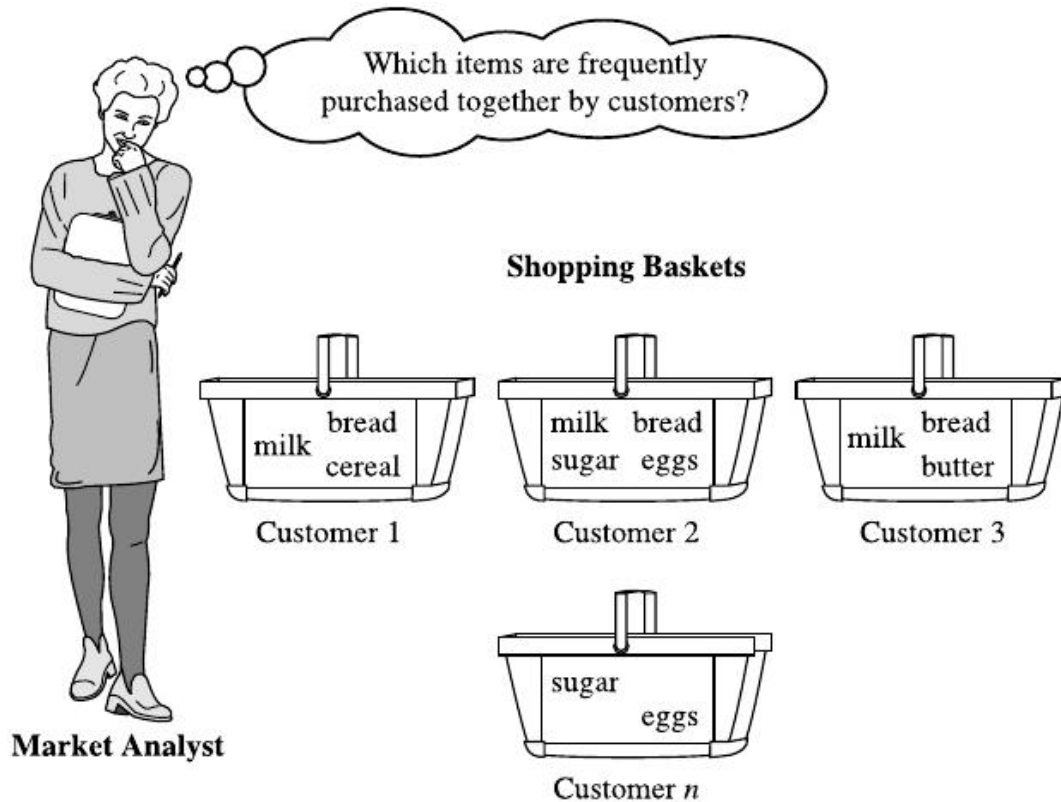
# 1. Basic Concepts

## 1.1. Market Basket Analysis: A Motivating Example

- A set of items is referred to as an **itemset**.
- Frequent itemset mining leads to the discovery of associations and correlations among items in large transactional or relational data sets.
- The discovery of interesting correlation relationships among huge amounts of business transaction records can help in many business decision-making processes such as catalog design, cross-marketing, and customer shopping behavior analysis.
- A typical example of frequent itemset mining is **market basket analysis**. This process analyzes customer buying habits by finding associations between the different items that customers place in their “shopping baskets”.

# 1. Basic Concepts

## 1.1. Market Basket Analysis: A Motivating Example



**FIGURE 4.1**

Market basket analysis.



# 1. Basic Concepts

## 1.1. Market Basket Analysis: A Motivating Example

- **Example 4.1. Market basket analysis.** Suppose, as manager of a retail company, you would like to learn more about the buying habits of your customers.
  - Specifically, you wonder, “*Which groups or sets of items are customers likely to purchase on a given trip to the store?*”
  - To answer your question, market basket analysis may be performed on the retail data of customer transactions at your store.
  - You can then use these results to choose marketing strategies and help create a new catalog. For instance, market basket analysis may help you design different store layouts.



# 1. Basic Concepts

## 1.1. Market Basket Analysis: A Motivating Example

- If we think of the universe as the set of items available at the store, then each item has a Boolean variable representing the presence or absence of that item.
- Each basket can then be represented by a Boolean vector of values assigned to these variables.
- The Boolean vectors can be analyzed to extract buying patterns that reflect items that are frequently *associated* or purchased together.
- These patterns can be represented in the form of **association rules**.
  - For example, the information that customers who purchase computers also tend to buy antivirus software at the same time is represented in the following association rule:

*computer*  $\Rightarrow$  *antivirus\_software* [*support* = 2%, *confidence* = 60%].



# 1. Basic Concepts

## 1.1. Market Basket Analysis: A Motivating Example

*computer*  $\Rightarrow$  *antivirus\_software* [*support* = 2%, *confidence* = 60%].

- Rule **support** and **confidence** are two measures of rule interestingness. They reflect the *usefulness* and *certainty* of discovered rules, respectively.
  - A support of 2% means that 2% of all the transactions under analysis show that computer and antivirus software are purchased together.
  - A confidence of 60% means that 60% of the customers who purchased a computer also bought the software.
- Typically, association rules are considered interesting if they satisfy a **minimum support threshold** and a **minimum confidence threshold**.
  - These thresholds can be set by users or domain experts.
- Additional analysis can be performed to discover interesting statistical correlations between associated items.



# 1. Basic Concepts

## 1.2. Frequent Itemsets, Closed ItemSets, and Association Rules

- Let  $I = \{I_1, I_2, \dots, I_m\}$  be an itemset. Let  $D$ , the task-relevant data, be a set of database transactions where each transaction  $T$  is a nonempty itemset such that  $T \subseteq I$ .
- Each transaction is associated with an identifier, called a *TID*. Let  $A$  be a set of items. A transaction  $T$  is said to contain  $A$  if  $A \subseteq T$ .
- An association rule is an implication of the form  $A \Rightarrow B$ , where  $A \subset I$ ,  $B \subset I$ ,  $A \neq \emptyset$ ,  $B \neq \emptyset$ , and  $A \cap B = \emptyset$ .
- The rule  $A \Rightarrow B$  holds in the transaction set  $D$  with support  $s$ , where  $s$  is the percentage of transactions in  $D$  that contain  $A \cup B$ . This is taken to be the probability,  $P(A \cup B)$ .
- The rule  $A \Rightarrow B$  has confidence  $c$  in the transaction set  $D$ , where  $c$  is the percentage of transactions in  $D$  containing  $A$  that also contain  $B$ . This is taken to be the conditional probability,  $P(B|A)$ . That is,

$$\text{support}(A \Rightarrow B) = P(A \cup B)$$

$$\text{confidence}(A \Rightarrow B) = P(B|A).$$

- Rules that satisfy both a minimum support threshold ( $\text{min\_sup}$ ) and a minimum confidence threshold ( $\text{min\_conf}$ ) are called **strong**. By convention, support and confidence values are represented as percentages.

# 1. Basic Concepts

## 1.2. Frequent Itemsets, Closed ItemSets, and Association Rules

- An itemset that contains  $k$  items is a  **$k$ -itemset**.
  - The set  $\{computer, antivirus\_software\}$  is a 2-itemset.
- The **occurrence frequency** of an itemset is the number of transactions that contain the itemset.
  - Occurrence frequency is also referred as the **frequency**, **support count**, or **count** of the itemset.
  - Note that the itemset support  $s$  is sometimes referred to as *relative support*, whereas the occurrence frequency is called the **absolute support**.
  - If the relative support of an itemset  $I$  satisfies a prespecified minimum support threshold (i.e., the absolute support of  $I$  satisfies the corresponding minimum support count threshold), then  $I$  is a **frequent** itemset.
  - The set of frequent  $k$ -itemsets is commonly denoted by  $L_k$ .

# 1. Basic Concepts

## 1.2. Frequent Itemsets, Closed ItemSets, and Association Rules

$$\text{confidence}(A \Rightarrow B) = P(B|A) = \frac{\text{support}(A \cup B)}{\text{support}(A)} = \frac{\text{support\_count}(A \cup B)}{\text{support\_count}(A)}.$$

- In general, association rule mining can be viewed as a two-step process:
  - **1. Find all frequent itemsets.** By definition, each of these itemsets will occur at least as frequently as a predetermined minimum support count, *min\_sup*.
  - **2. Generate strong association rules from the frequent itemsets.** By definition, these rules must satisfy minimum support and minimum confidence.

# 1. Basic Concepts

## 1.2. Frequent Itemsets, Closed ItemSets, and Association Rules

- A major challenge in mining frequent itemsets from a large data set is the fact that such mining often generates a huge number of itemsets satisfying the minimum support ( $min\_sup$ ) threshold, especially when  $min\_sup$  is set low.
- This is because if an itemset is frequent, each of its subsets is frequent as well. A long itemset will contain a combinatorial number of shorter frequent subitemsets.
  - For example, a frequent itemset of length 100, such as  $\{a_1, a_2, \dots, a_{100}\}$ , contains
    - $\binom{100}{1} = 100$  frequent 1-itemsets:  $\{a_1\}, \{a_2\}, \dots, \{a_{100}\}$ ;
    - $\binom{100}{2}$  frequent 2-itemsets:  $\{a_1, a_2\}, \{a_1, a_3\}, \{a_1, a_4\}, \dots, \{a_2, a_3\}, \{a_2, a_4\}, \dots, \{a_{99}, a_{100}\}$ ; and so on.
    - The total number of frequent itemsets that it contains is thus
    - $\binom{100}{1} + \binom{100}{2} + \dots + \binom{100}{100} = 2^{100} - 1 \approx 1.27 \times 10^{30}$ .
  - This is too huge a number of itemsets for any computer to compute or store. To overcome this difficulty, we introduce the concepts of *closed frequent itemset* and *maximal frequent itemset*.

# 1. Basic Concepts

## 1.2. Frequent Itemsets, Closed ItemSets, and Association Rules

- $Y$  is a proper superitemset of  $X$  if  $X$  is a proper subitemset of  $Y$ , that is, if  $X \subset Y$ .
  - In other words, every item of  $X$  is contained in  $Y$ , but there is at least one item of  $Y$  that is not in  $X$ .
- An itemset  $X$  is **closed** in a data set  $D$  if there exists no proper superitemset  $Y$  such that  $Y$  has the same support count as  $X$  in  $D$ .
- An itemset  $X$  is a **closed frequent itemset** in set  $D$  if  $X$  is both closed and frequent in  $D$ .
- An itemset  $X$  is a **maximal frequent itemset** (or **max-itemset**) in a data set  $D$  if  $X$  is frequent, and there exists no superitemset  $Y$  such that  $X \subset Y$  and  $Y$  is frequent in  $D$ .

# 1. Basic Concepts

## 1.2. Frequent Itemsets, Closed ItemSets, and Association Rules

- Let  $C$  be the set of closed frequent itemsets for a data set  $D$  satisfying a minimum support threshold,  $min\_sup$ .
- Let  $M$  be the set of maximal frequent itemsets for  $D$  satisfying  $min\_sup$ .
- Suppose that we have the support count of each itemset in  $C$  and  $M$ .
- **Notice that  $C$  and its count information can be used to derive the whole set of frequent itemsets. Thus we say that  $C$  contains complete information regarding its corresponding frequent itemsets.**
- On the other hand,  $M$  registers only the support of the maximal itemsets. It usually does not contain the complete support information regarding its corresponding frequent itemsets.

# 1. Basic Concepts

## 1.2. Frequent Itemsets, Closed ItemSets, and Association Rules

- **Example 4.2. Closed and maximal frequent itemsets.** Suppose that a transaction database has only two transactions:  $\{ \langle a_1, a_2, \dots, a_{100} \rangle ; \langle a_1, a_2, \dots, a_{50} \rangle \}$ .
  - Let the minimum support count threshold be  $min\_sup = 1$ .
  - We find two closed frequent itemsets and their support counts, that is,  $C = \{ \{a_1, a_2, \dots, a_{100}\} : 1; \{a_1, \dots, a_{50}\} : 2 \}$ .
  - There is only one maximal frequent itemset:  $M = \{ \{a_1, a_2, \dots, a_{100}\} : 1 \}$ . Notice that we cannot include  $\{a_1, a_2, \dots, a_{50}\}$  as a maximal frequent itemset because it has a frequent superset,  $\{a_1, a_2, \dots, a_{100}\}$ .
  - Compare this to the preceding where we determined that there are  $2^{100} - 1$  frequent itemsets, which are too many to be enumerated!

# 1. Basic Concepts

## 1.2. Frequent Itemsets, Closed ItemSets, and Association Rules

- The set of closed frequent itemsets contains complete information regarding the frequent itemsets.
  - For example, from  $C$ , we can derive, say,
    - (1)  $\{a_2, a_{45}\}: 2$  since  $\{a_2, a_{45}\}$  is a subitemset of the itemset  $\{a_1, \dots, a_{50}\}: 2$
    - (2)  $\{a_8, a_{55}\}: 1$  since  $\{a_8, a_{55}\}$  is not a subitemset of the previous itemset but of the itemset  $\{a_1, a_2, \dots, a_{100}\}: 1$ .
    - However, from the maximal frequent itemset, we can only assert that both itemsets ( $\{a_2, a_{45}\}$  and  $\{a_8, a_{55}\}$ ) are frequent, but we cannot assert their actual support counts.



## 2. Frequent Itemset Mining Methods

### 2.1 Apriori Algorithm: Finding Frequent Itemsets by Confined Candidate Generation

- The name of the algorithm is based on the fact that the algorithm uses *prior knowledge* of frequent itemset properties, as we shall see later.
- **Apriori** employs an iterative approach known as a *level-wise* search, where  $k$ -itemsets are used to explore  $(k + 1)$ -itemsets.
  - First, the set of frequent 1-itemsets is found by scanning the database to accumulate the count for each item, and collecting those items that satisfy minimum support. The resulting set is denoted by  $L_1$ .
  - Next,  $L_1$  is used to find  $L_2$ , the set of frequent 2-itemsets, which is used to find  $L_3$ , and so on, until no more frequent  $k$ -itemsets can be found.
  - The finding of each  $L_k$  requires one full scan of the database.
- To improve the efficiency of the level-wise generation of frequent itemsets, an important property called the **Apriori property** is used to reduce the search space.

## 2. Frequent Itemset Mining Methods

### 2.1 Apriori Algorithm: Finding Frequent Itemsets by Confined Candidate Generation

- **Apriori property:** All nonempty subsets of a frequent itemset must also be frequent.
- How  $L_{k-1}$  is used to find  $L_k$  for  $k \geq 2$ . A two-step process is followed, consisting of **join** and **prune** actions.

## 2. Frequent Itemset Mining Methods

### 2.1 Apriori Algorithm: Finding Frequent Itemsets by Confined Candidate Generation

- **1. The join step.**
  - To find  $L_k$ , a set of **candidate**  $k$ -itemsets is generated by joining  $L_{k-1}$  with itself. This set of candidates is denoted  $C_k$ .
  - Let  $l_1$  and  $l_2$  be itemsets in  $L_{k-1}$ . The notation  $l_i[j]$  refers to the  $j$ th item in  $l_i$ .
  - For efficient implementation, Apriori assumes that items within a transaction or itemset are sorted in lexicographic order. For the  $(k-1)$ -itemset,  $l_i$ , this means that the items are sorted such that  $l_i[1] < l_i[2] < \dots < l_i[k-1]$ .
  - The join,  $L_{k-1} \bowtie L_{k-1}$ , is performed, where **members of  $L_{k-1}$  are joinable if their first  $(k-2)$  items are in common**. That is, members  $l_1$  and  $l_2$  of  $L_{k-1}$  are joined if  $(l_1[1] = l_2[1]) \wedge (l_1[2] = l_2[2]) \wedge \dots \wedge (l_1[k-2] = l_2[k-2]) \wedge (l_1[k-1] < l_2[k-1])$ .
    - The condition  $l_1[k-1] < l_2[k-1]$  simply ensures that no duplicates are generated.
  - The resulting itemset formed by joining  $l_1$  and  $l_2$  is  $\{l_1[1], l_1[2], \dots, l_1[k-2], l_1[k-1], l_2[k-1]\}$ .

## 2. Frequent Itemset Mining Methods

### 2.1 Apriori Algorithm: Finding Frequent Itemsets by Confined Candidate Generation

- **2. The prune step.**

- $C_k$  is a superset of  $L_k$ , that is, its members may or may not be frequent. A database scan to determine the count of each candidate in  $C_k$  would result in the determination of  $L_k$  (i.e., all candidates having a count no less than the minimum support count are frequent by definition and therefore belong to  $L_k$ ).
- $C_k$ , however, can be huge, and so this could involve heavy computation. To reduce the size of  $C_k$ , the Apriori property is used as follows.
- Any  $(k-1)$ -itemset that is not frequent cannot be a subset of a frequent  $k$ -itemset. Hence, if any  $(k-1)$ -subset of a candidate  $k$ -itemset is not in  $L_{k-1}$ , then the candidate cannot be frequent either and so can be removed from  $C_k$ . This **subset testing** can be done quickly by maintaining a hash tree of all frequent itemsets.

## 2. Frequent Itemset Mining Methods

### 2.1 Apriori Algorithm: Finding Frequent Itemsets by Confined Candidate Generation

- **Example 4.3. Apriori.** Let's look at a concrete example, based on the transaction database,  $D$ , of Table 4.1. There are nine transactions in this database, that is,  $|D| = 9$ . We use Fig. 4.2 to illustrate the Apriori algorithm for finding frequent itemsets in  $D$ .

**Table 4.1** A transactional data set.

| TID  | List of item_IDs |
|------|------------------|
| T100 | I1, I2, I5       |
| T200 | I2, I4           |
| T300 | I2, I3           |
| T400 | I1, I2, I4       |
| T500 | I1, I3           |
| T600 | I2, I3           |
| T700 | I1, I3           |
| T800 | I1, I2, I3, I5   |
| T900 | I1, I2, I3       |

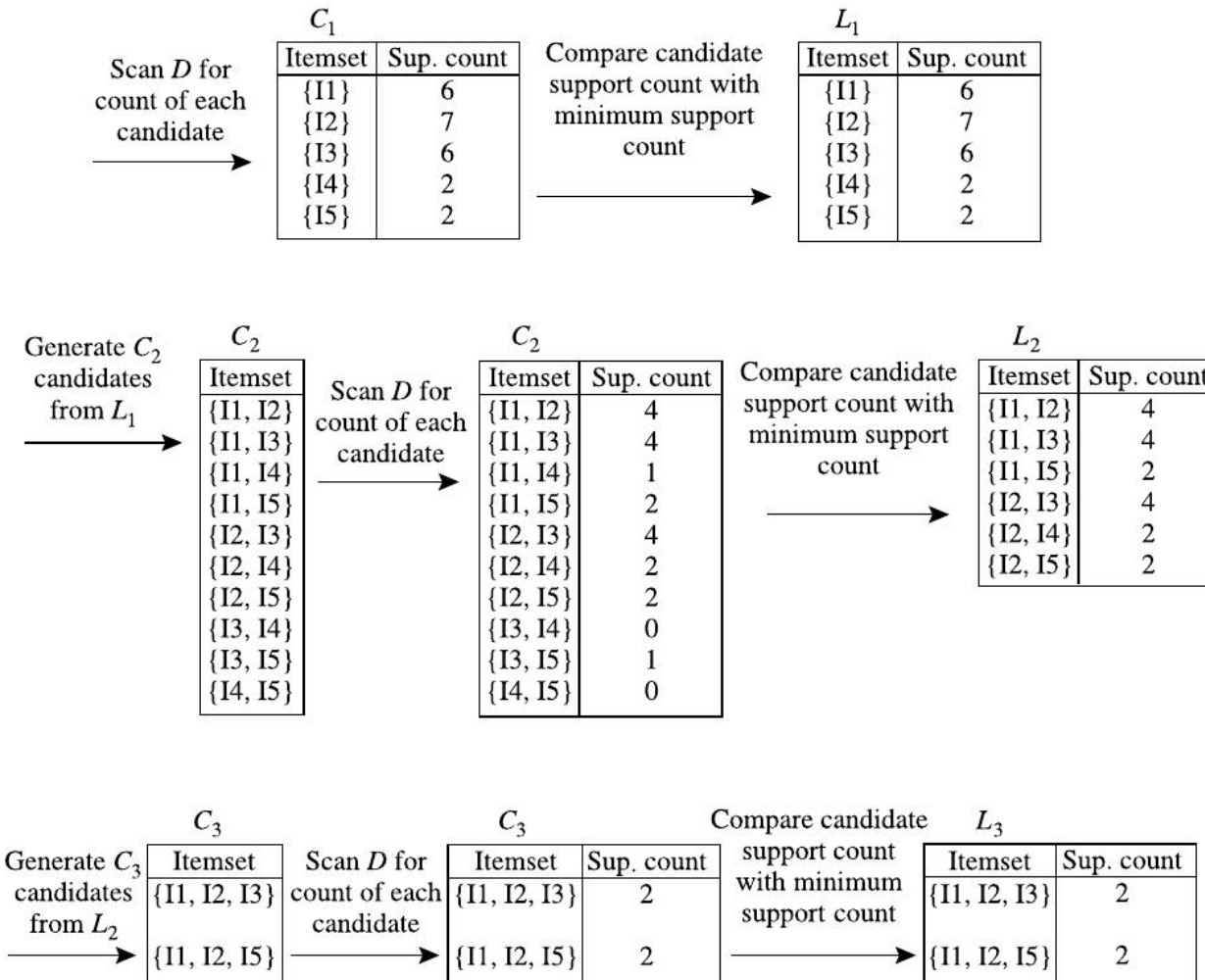
## 2. Frequent Itemset Mining Methods

### 2.1 Apriori Algorithm: Finding Frequent Itemsets by Confined Candidate Generation

**Table 4.1 A transactional data set.**

| TID  | List of item_IDs |
|------|------------------|
| T100 | I1, I2, I5       |
| T200 | I2, I4           |
| T300 | I2, I3           |
| T400 | I1, I2, I4       |
| T500 | I1, I3           |
| T600 | I2, I3           |
| T700 | I1, I3           |
| T800 | I1, I2, I3, I5   |
| T900 | I1, I2, I3       |

- The algorithm uses  $L_3 \bowtie L_3$  to generate a candidate set of 4-itemsets,  $C_4$ . Although the join results in  $\{\{I_1, I_2, I_3, I_5\}\}$ , itemset  $\{I_1, I_2, I_3, I_5\}$  is pruned because its subset  $\{I_2, I_3, I_5\}$  is not frequent.
- Thus,  $C_4 = \emptyset$ , and the algorithm terminates, having found all of the frequent itemsets.



**FIGURE 4.2**

Generation of the candidate itemsets and frequent itemsets, where the minimum support count is 2.



## 2.1 Apriori Algorithm: Finding Frequent Itemsets by Confined Candidate Generation

| <b>TID</b> | <b>List of item_IDs</b> |
|------------|-------------------------|
| T100       | I1, I2, I5              |
| T200       | I2, I4                  |
| T300       | I2, I3                  |
| T400       | I1, I2, I4              |
| T500       | I1, I3                  |
| T600       | I2, I3                  |
| T700       | I1, I3                  |
| T800       | I1, I2, I3, I5          |
| T900       | I1, I2, I3              |

- | Itemset  | Sup. count |
|----------|------------|
| {I1, I2} | 4          |
| {I1, I3} | 4          |
| {I1, I5} | 2          |
| {I2, I3} | 4          |
| {I2, I4} | 2          |
| {I2, I5} | 2          |

Generation and pruning of candidate 3-itemsets,  $C_3$ , from  $L_2$  using the Apriori property.



Generation of the candidate itemsets and frequent itemsets, where the minimum support count is 2.



## 2. Frequent Itemset Mining Methods

### 2.2 Generating Association Rules From Frequent Itemsets

- Once the frequent itemsets from transactions in a database  $D$  have been found, it is straightforward to generate strong association rules from them (where strong association rules satisfy both minimum support and minimum confidence).

$$\text{confidence}(A \Rightarrow B) = P(B|A) = \frac{\text{support}(A \cup B)}{\text{support}(A)} = \frac{\text{support\_count}(A \cup B)}{\text{support\_count}(A)}.$$

- For each frequent itemset  $l$ , generate all nonempty subsets of  $l$ .
- For every nonempty subset  $s$  of  $l$ , output the rule “ $s \Rightarrow (l - s)$ ” if

$$\frac{\text{support\_count}(l)}{\text{support\_count}(s)} \geq \text{min\_conf},$$

where  $\text{min\_conf}$  is the minimum confidence threshold.





## 2. Frequent Itemset Mining Methods

### 2.2 Generating Association Rules From Frequent Itemsets

- **Example 4.4. Generating association rules.** Let's try an example based on the transactional data shown before in Table 4.1. The data contain frequent itemset  $X = \{I_1, I_2, I_5\}$ . What are the association rules that can be generated from  $X$ ?
  - The nonempty subsets of  $X$  are  $\{I_1, I_2\}$ ,  $\{I_1, I_5\}$ ,  $\{I_2, I_5\}$ ,  $\{I_1\}$ ,  $\{I_2\}$ , and  $\{I_5\}$ .
  - The resulting association rules are as shown below, each listed with its confidence:
    - $\{I_1, I_2\} \Rightarrow I_5$ , *confidence* =  $2/4 = 50\%$
    - $\{I_1, I_5\} \Rightarrow I_2$ , *confidence* =  $2/2 = 100\%$
    - $\{I_2, I_5\} \Rightarrow I_1$ , *confidence* =  $2/2 = 100\%$
    - $\{I_1\} \Rightarrow \{I_2, I_5\}$ , *confidence* =  $2/6 = 33\%$
    - $\{I_2\} \Rightarrow \{I_1, I_5\}$ , *confidence* =  $2/7 = 29\%$
    - $\{I_5\} \Rightarrow \{I_1, I_2\}$ , *confidence* =  $2/2 = 100\%$
  - If the minimum confidence threshold is 70%, then only the second, third, and last rules are output, because these are the only ones generated that are strong.



## 2. Frequent Itemset Mining Methods

### 2.3 Improving the Efficiency of Apriori

- Read pages 155-157



## 2. Frequent Itemset Mining Methods

### 2.4 A Pattern-Growth Approach for Mining Frequent Itemsets

- In many cases the Apriori candidate generate-and-test method significantly reduces the size of candidate sets, leading to good performance gain. However, it can suffer from two nontrivial costs.
  - *It may still need to generate a huge number of candidate sets.* For example, if there are  $10^4$  frequent 1-itemsets, the Apriori algorithm will need to generate more than  $10^7$  candidate 2-itemsets.
  - *It may need to repeatedly scan the whole database and check a large set of candidates by pattern matching.* It is costly to go over each transaction in the database to determine the support of the candidate itemsets.
- An interesting method in this attempt is called **frequent pattern growth (FP-growth)** which adopts a *divide-and-conquer* strategy as follows.



## 2. Frequent Itemset Mining Methods

### 2.4 A Pattern-Growth Approach for Mining Frequent Itemsets

- Frequent Pattern Growth
  - It compresses the database representing frequent items into a **frequent pattern tree (FP-tree)** which retains the itemset association information.
  - It then divides the compressed database into a set of *conditional databases*, each associated with one itemset found so far, or “pattern fragment,” and mines each database separately.
  - For each “pattern fragment,” only its associated data sets need to be examined. Therefore this approach may substantially reduce the size of the data sets to be searched, along with the “growth” of patterns being examined.

## 2. Frequent Itemset Mining Methods

### 2.4 A Pattern-Growth Approach for Mining Frequent Itemsets

- **Example 4.5. FP-growth (finding frequent itemsets without candidate generation).** We reexamine the mining of transaction database,  $D$ , of Table 4.1.

**Table 4.1** A transactional data set.

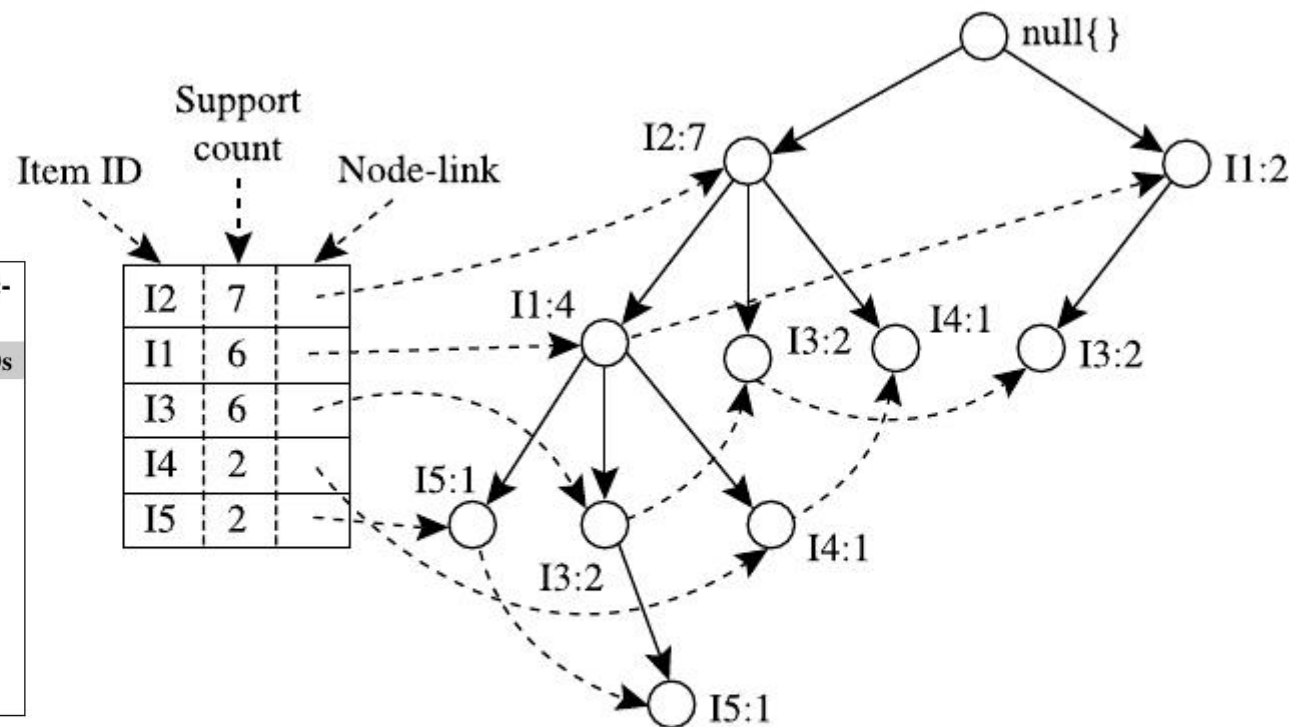
| TID  | List of item_IDs |
|------|------------------|
| T100 | I1, I2, I5       |
| T200 | I2, I4           |
| T300 | I2, I3           |
| T400 | I1, I2, I4       |
| T500 | I1, I3           |
| T600 | I2, I3           |
| T700 | I1, I3           |
| T800 | I1, I2, I3, I5   |
| T900 | I1, I2, I3       |

- The first scan of the database is the same as Apriori, which derives the set of frequent items (1-itemsets) and their support counts (frequencies).
- Let the minimum support count be 2.
- The set of frequent items is sorted in the order of descending support count. This resulting set or list is denoted by  $L$ .
  - Thus, we have  $L = \{\{I_2: 7\}, \{I_1: 6\}, \{I_3: 6\}, \{I_4: 2\}, \{I_5: 2\}\}$ .
- An FP-tree is then constructed as follows. First, create the root of the tree, labeled with “null.” Scan database  $D$  a second time. The items in each transaction are processed in  $L$  order (i.e., sorted according to descending support count), and a branch is created for each transaction.

## 2. Frequent Itemset Mining Methods

### 2.4 A Pattern-Growth Approach for Mining Frequent Itemsets

| Table 4.1 A transactional data set. |                  |
|-------------------------------------|------------------|
| TID                                 | List of item_IDs |
| T100                                | I1, I2, I5       |
| T200                                | I2, I4           |
| T300                                | I2, I3           |
| T400                                | I1, I2, I4       |
| T500                                | I1, I3           |
| T600                                | I2, I3           |
| T700                                | I1, I3           |
| T800                                | I1, I2, I3, I5   |
| T900                                | I1, I2, I3       |



- The scan of the first transaction, “T100: I1, I2, I5,” which contains three items (I2, I1, I5 in *L* order), leads to the construction of the first branch of the tree with three nodes,  $\langle I2: 1 \rangle$ ,  $\langle I1: 1 \rangle$ , and  $\langle I5: 1 \rangle$ , where I2 is linked as a child to the root, I1 is linked to I2, and I5 is linked to I1.

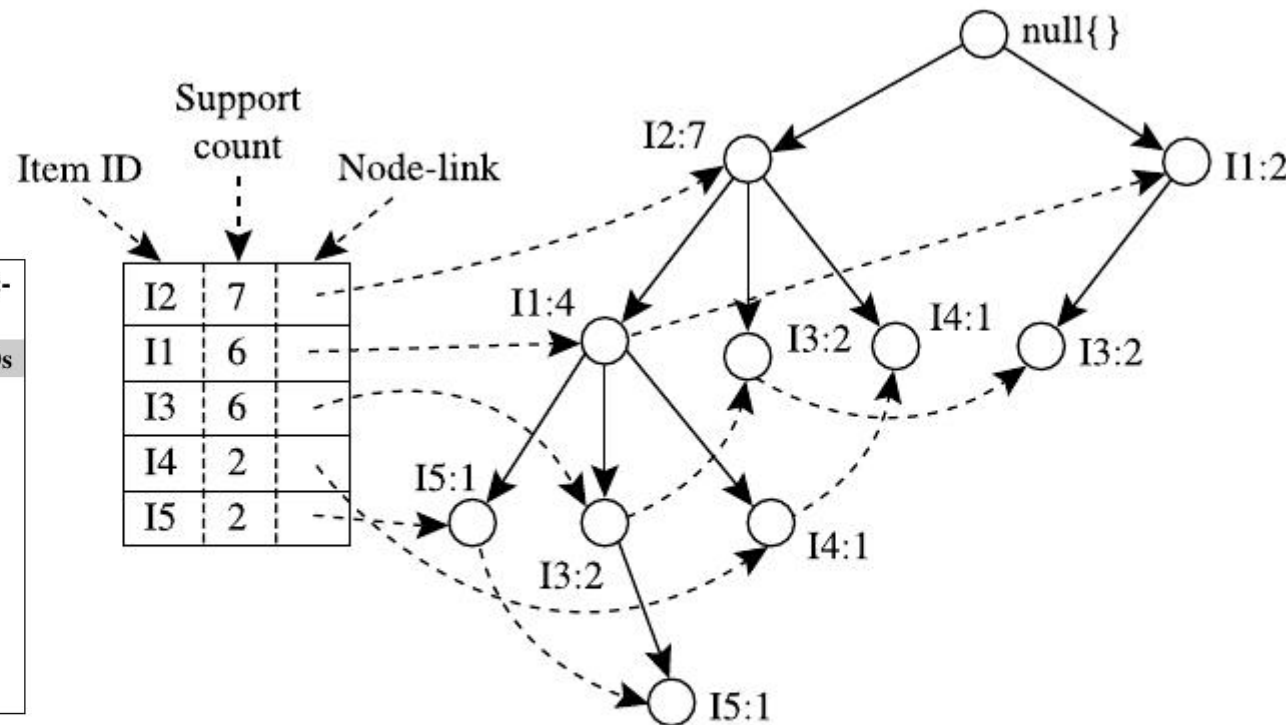
FIGURE 4.7

An FP-tree registers compressed frequent pattern information.

## 2. Frequent Itemset Mining Methods

### 2.4 A Pattern-Growth Approach for Mining Frequent Itemsets

| Table 4.1 A transactional data set. |                  |
|-------------------------------------|------------------|
| TID                                 | List of item_IDs |
| T100                                | I1, I2, I5       |
| T200                                | I2, I4           |
| T300                                | I2, I3           |
| T400                                | I1, I2, I4       |
| T500                                | I1, I3           |
| T600                                | I2, I3           |
| T700                                | I1, I3           |
| T800                                | I1, I2, I3, I5   |
| T900                                | I1, I2, I3       |



- The second transaction, T200, contains the items I2 and I4 in  $L$  order, which would result in a branch where I2 is linked to the root and I4 is linked to I2.
- However, this branch would share a common **prefix**, I2, with the existing path for T100. Therefore, we instead increment the count of the I2 node by 1, and create a new node,  $\langle I4: 1 \rangle$ , which is linked as a child to  $\langle I2: 2 \rangle$ .
- In general, when considering the branch to be added for a transaction, the count of each node along a common prefix is incremented by 1, and nodes for the items following the prefix are created and linked accordingly.

**FIGURE 4.7**

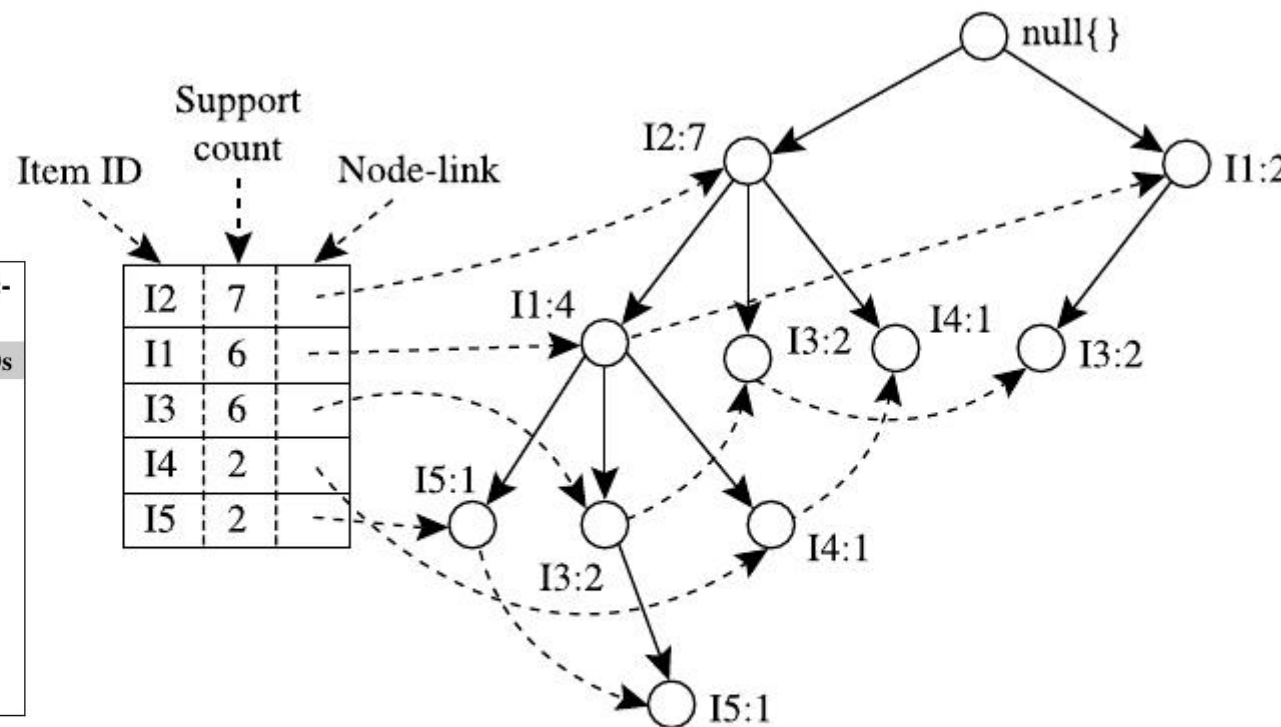
An FP-tree registers compressed frequent pattern information.



## 2. Frequent Itemset Mining Methods

### 2.4 A Pattern-Growth Approach for Mining Frequent Itemsets

| Table 4.1 A transactional data set. |                  |
|-------------------------------------|------------------|
| TID                                 | List of item_IDs |
| T100                                | I1, I2, I5       |
| T200                                | I2, I4           |
| T300                                | I2, I3           |
| T400                                | I1, I2, I4       |
| T500                                | I1, I3           |
| T600                                | I2, I3           |
| T700                                | I1, I3           |
| T800                                | I1, I2, I3, I5   |
| T900                                | I1, I2, I3       |



- To facilitate tree traversal, an item header table is built so that each item points to its occurrences in the tree via a chain of **node-links**.
- The problem of mining frequent patterns in databases is transformed into that of mining the FP-tree.

**FIGURE 4.7**

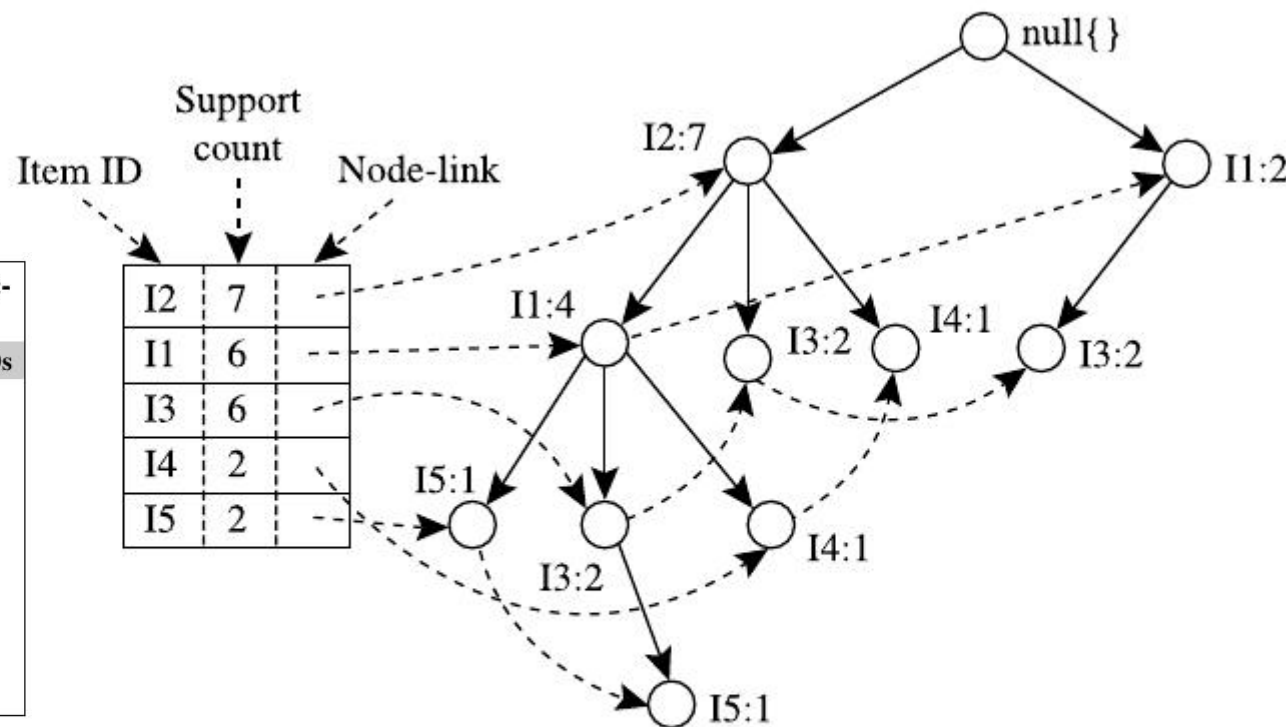
An FP-tree registers compressed frequent pattern information.



## 2. Frequent Itemset Mining Methods

### 2.4 A Pattern-Growth Approach for Mining Frequent Itemsets

| Table 4.1 A transactional data set. |                  |
|-------------------------------------|------------------|
| TID                                 | List of item_IDs |
| T100                                | I1, I2, I5       |
| T200                                | I2, I4           |
| T300                                | I2, I3           |
| T400                                | I1, I2, I4       |
| T500                                | I1, I3           |
| T600                                | I2, I3           |
| T700                                | I1, I3           |
| T800                                | I1, I2, I3, I5   |
| T900                                | I1, I2, I3       |



- The FP-tree is mined as follows. Start from each frequent length-1 pattern (as an initial **suffix pattern**), construct its **conditional pattern base** (a “subdatabase,” which consists of the set of *prefix paths* in the FP-tree cooccurring with the suffix pattern), then construct its (*conditional*) FP-tree, and perform mining recursively on the tree. The pattern growth is achieved by the concatenation of the suffix pattern with the frequent patterns generated from a conditional FP-tree.

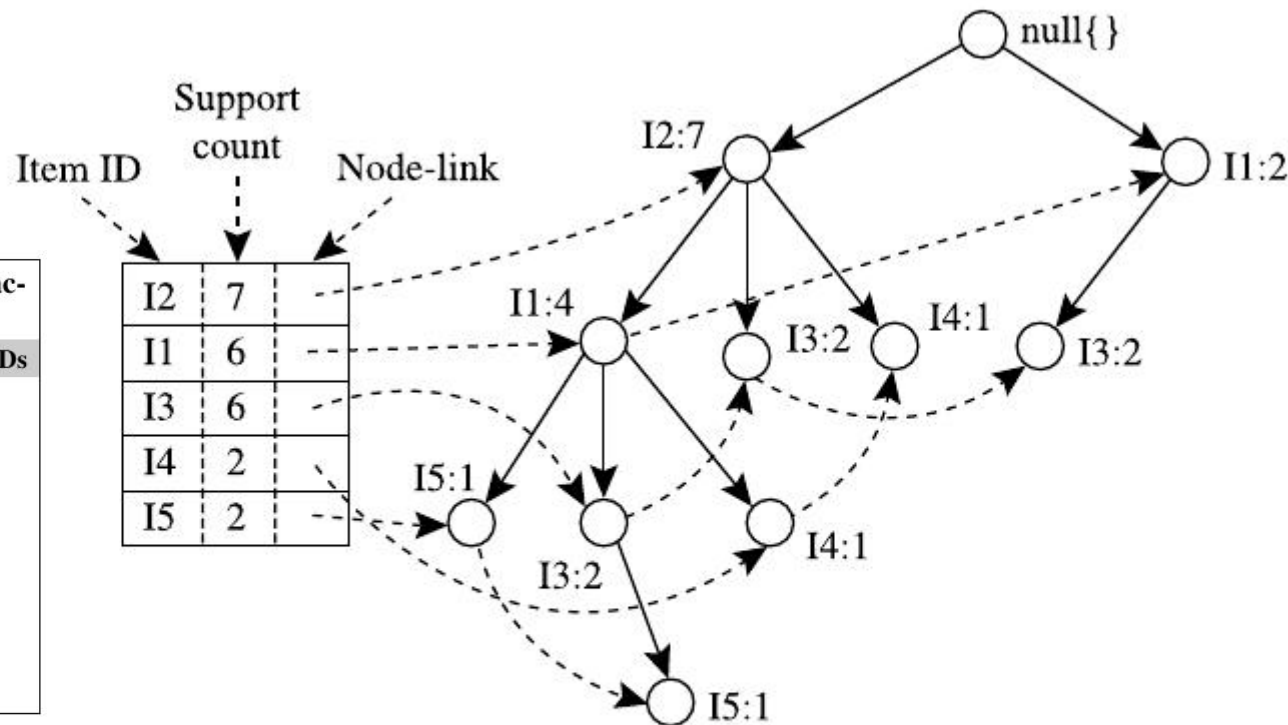
**FIGURE 4.7**

An FP-tree registers compressed frequent pattern information.

## 2. Frequent Itemset Mining Methods

### 2.4 A Pattern-Growth Approach for Mining Frequent Itemsets

| Table 4.1 A transactional data set. |                  |
|-------------------------------------|------------------|
| TID                                 | List of item_IDs |
| T100                                | I1, I2, I5       |
| T200                                | I2, I4           |
| T300                                | I2, I3           |
| T400                                | I1, I2, I4       |
| T500                                | I1, I3           |
| T600                                | I2, I3           |
| T700                                | I1, I3           |
| T800                                | I1, I2, I3, I5   |
| T900                                | I1, I2, I3       |



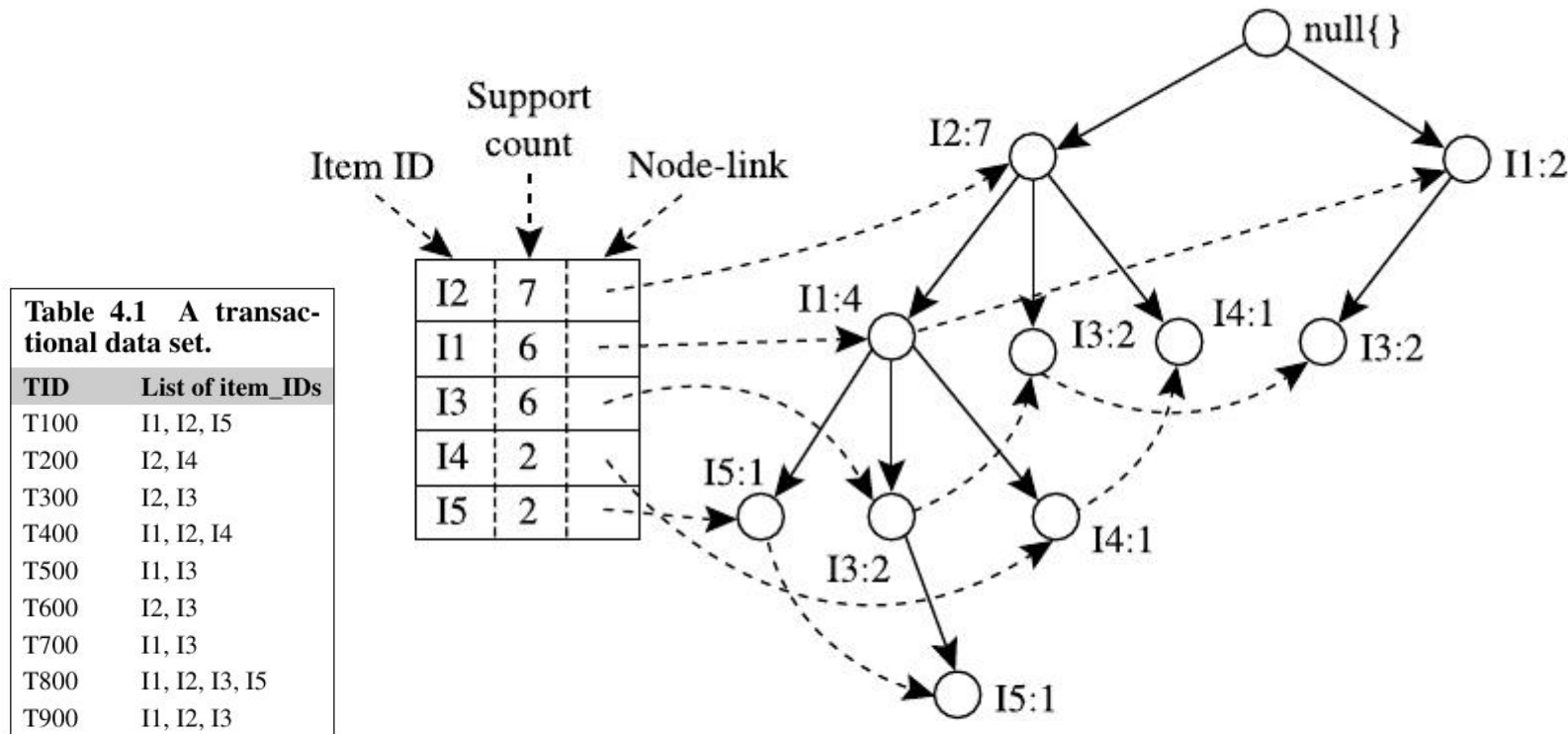
- We first consider I5, which is the last item in  $L$ , rather than the first.
- I5: Suffix
  - Two prefix paths:  $\langle I2, I1, I5: 1 \rangle$ ,  $\langle I2, I1, I3, I5: 1 \rangle$ , which form its conditional pattern base.
  - Using this conditional pattern base as a transaction database, we build an I5-conditional FP-tree, which contains only a single path,  $\langle I2: 2, I1: 2 \rangle$ ; I3 is not included because its support count of 1 is less than the minimum support count.
  - Combinations of frequent patterns:  $\{I2, I5: 2\}$ ,  $\{I1, I5: 2\}$ ,  $\{I2, I1, I5: 2\}$ .

FIGURE 4.7

An FP-tree registers compressed frequent pattern information.

## 2. Frequent Itemset Mining Methods

### 2.4 A Pattern-Growth Approach for Mining Frequent Itemsets



**FIGURE 4.7**

An FP-tree registers compressed frequent pattern

**Table 4.2 Mining the FP-tree by creating conditional (sub-)pattern bases.**

| Item | Conditional Pattern Base          | Conditional FP-tree                                   | Frequent Patterns Generated               |
|------|-----------------------------------|-------------------------------------------------------|-------------------------------------------|
| I5   | { {I2, I1: 1}, {I2, I1, I3: 1} }  | $\langle I2: 2, I1: 2 \rangle$                        | {I2, I5: 2}, {I1, I5: 2}, {I2, I1, I5: 2} |
| I4   | { {I2, I1: 1}, {I2: 1} }          | $\langle I2: 2 \rangle$                               | {I2, I4: 2}                               |
| I3   | { {I2, I1: 2}, {I2: 2}, {I1: 2} } | $\langle I2: 4, I1: 2 \rangle, \langle I1: 2 \rangle$ | {I2, I3: 4}, {I1, I3: 4}, {I2, I1, I3: 2} |
| I1   | { {I2: 4} }                       | $\langle I2: 4 \rangle$                               | {I2, I1: 4}                               |



### 3. Which Patterns Are Interesting? Pattern Evaluation Methods

- Most association rule mining algorithms employ a support – confidence framework.
- Although minimum support and confidence thresholds help weed out or exclude the exploration of a good number of uninteresting rules, many of the rules generated are still not interesting to many users.
  - This is especially true when mining at low support thresholds or mining for long patterns. This has been a major bottleneck for successful application of association rule mining.



## 3. Which Patterns Are Interesting? Pattern Evaluation Methods

### 3.1. Strong Rules Are Not Necessarily Interesting

- The interestingness of a rule can be assessed either subjectively or objectively.
- Ultimately, only the user can judge if a given rule is interesting, and this judgment, being subjective, may differ from one user to another.
- However, objective interestingness measures, based on the statistics “behind” the data, can be used as one step toward the goal of weeding out uninteresting rules that would otherwise be presented to the user.
- *“How can we tell which strong association rules are really interesting?”*



## 3. Which Patterns Are Interesting? Pattern Evaluation Methods

### 3.1. Strong Rules Are Not Necessarily Interesting

- **Example 4.7. A misleading “strong” association rule.** Suppose we are interested in analyzing transactions with respect to the purchase of computer games and videos.
- Let *game* refer to the transactions containing computer games, and *video* refer to those containing videos.
- Of the 10,000 transactions analyzed, the data show that 6000 of the customer transactions included computer games, whereas 7500 included videos, and 4000 included both computer games and videos.
- Suppose that a data mining program for discovering association rules is run on the data, using a minimum support of 30% and a minimum confidence of 60%. The following association rule is discovered:

$$\begin{aligned} & \text{buys}(X, \text{“computer games”}) \Rightarrow \text{buys}(X, \text{“videos”}) \\ & [\text{support} = 40\%, \text{confidence} = 66\%]. \end{aligned} \tag{4.6}$$





## 3. Which Patterns Are Interesting? Pattern Evaluation Methods

### 3.1. Strong Rules Are Not Necessarily Interesting

$$\begin{aligned} & \text{buys}(X, \text{"computer games"}) \Rightarrow \text{buys}(X, \text{"videos"}) \\ & [\text{support} = 40\%, \text{confidence} = 66\%]. \end{aligned} \tag{4.6}$$

- Rule (4.6) is a strong association rule and would therefore be reported, since its support value of  $4000/10,000 = 40\%$  and confidence value of  $4000/6000 = 66\%$  satisfy the minimum support and minimum confidence thresholds, respectively.
- However, Rule (4.6) is misleading because the probability of purchasing videos is 75%, which is even larger than 66%. In fact, computer games and videos are negatively associated because the purchase of one of these items actually decreases the likelihood of purchasing the other.
- Without fully understanding this phenomenon, we could easily make unwise business decisions based on the rule.
- Example 4.7 also illustrates that the confidence of a rule  $A \Rightarrow B$  can be deceiving. It does not measure the *real strength* (or lack of strength) of the *correlation* and *implication* between  $A$  and  $B$ . Hence, alternatives to the support–confidence framework can be useful in mining interesting data relationships.



## 3. Which Patterns Are Interesting? Pattern Evaluation Methods

### 3.2. From Association Analysis to Correlation Analysis

- As we have seen so far, the support and confidence measures are insufficient at filtering out uninteresting association rules.
- To tackle this weakness, a correlation measure can be augmented to the support–confidence framework for association rules.
- This leads to correlation rules of the form

$$A \Rightarrow B [\textit{support}, \textit{confidence}, \textit{correlation}]. \quad (4.7)$$

- That is, a correlation rule is measured not only by its support and confidence but also by the correlation between itemsets  $A$  and  $B$ . There are many different correlation measures for us to choose. In this subsection, we study several correlation measures to determine which would be good for mining large data sets.





## 3. Which Patterns Are Interesting? Pattern Evaluation Methods

### 3.2. From Association Analysis to Correlation Analysis

- **Lift** is a simple correlation measure that is given as follows.
- The occurrence of itemset  $A$  is **independent** of the occurrence of itemset  $B$  if  $P(A \cup B) = P(A)P(B)$ ; otherwise, itemsets  $A$  and  $B$  are **dependent** and **correlated**.
- This definition can easily be extended to more than two itemsets.
- The **lift** between the occurrence of  $A$  and  $B$  can be measured by computing

$$\text{lift}(A, B) = \frac{P(A \cup B)}{P(A)P(B)}. \quad (4.8)$$

- If the resulting value of Eq. (4.8) is less than 1, then the occurrence of  $A$  is *negatively correlated* with the occurrence of  $B$ , meaning that the occurrence of one likely leads to the absence of the other one.
- If the resulting value is greater than 1, then  $A$  and  $B$  are *positively correlated*, meaning that the occurrence of one implies the occurrence of the other.
- If the resulting value is equal to 1, then  $A$  and  $B$  are independent, and there is no correlation between them.



### 3. Which Patterns Are Interesting? Pattern Evaluation Methods

#### 3.2. From Association Analysis to Correlation Analysis

- Remember  $confidence(A \Rightarrow B) = P(B|A) = \frac{support(A \cup B)}{support(A)} = \frac{support\_count(A \cup B)}{support\_count(A)}$ .
- $confidence(A \Rightarrow B) = P(B|A) = lift(A, B) \cdot P(B)$ 
  - $lift(A, B)$ : assesses the degree to which the occurrence of one lifts the occurrence of the other.
    - For example, if  $A$  corresponds to the sale of computer games and  $B$  corresponds to the sale of videos, then given the current market conditions, the sale of games is said to increase or “lift” the likelihood of the sale of videos by a factor of the value returned by  $lift(A, B)$ .

### 3. Which Patterns Are Interesting? Pattern Evaluation Methods

#### 3.2. From Association Analysis to Correlation Analysis

- **Example 4.8. Correlation analysis using lift.** To help filter out misleading “strong” associations of the form  $A \Rightarrow B$  from the data of Example 4.7, we need to study how the two itemsets,  $A$  and  $B$ , are correlated.
- Let  $\overline{game}$  refer to the transactions of Example 4.7 that do not contain computer games, and  $\overline{video}$  refer to those that do not contain videos. The transactions can be summarized in a *contingency table*, as shown in Table 4.6.

**Table 4.6**  $2 \times 2$  contingency table summarizing the transactions with respect to game and video purchases.

|                    | <i>game</i> | $\overline{game}$ | $\Sigma_{row}$ |
|--------------------|-------------|-------------------|----------------|
| <i>video</i>       | 4000        | 3500              | 7500           |
| $\overline{video}$ | 2000        | 500               | 2500           |
| $\Sigma_{col}$     | 6000        | 4000              | 10,000         |

### 3. Which Patterns Are Interesting? Pattern Evaluation Methods

#### 3.2. From Association Analysis to Correlation Analysis

- $P(\{game\}) = 0.60$
- $P(\{video\}) = 0.75$
- $P(\{game, video\}) = 0.40$

- $lift(game, video) = \frac{P(\{game, video\})}{P(\{game\})P(\{video\})} = \frac{0.40}{0.60 \times 0.75} = 0.89$

- Because this value is less than 1, there is a negative correlation between the occurrence of  $\{game\}$  and  $\{video\}$ . Such a negative correlation cannot be identified by a support–confidence framework.

**Table 4.6** 2 × 2 contingency table summarizing the transactions with respect to game and video purchases.

|                    | <i>game</i> | $\overline{game}$ | $\Sigma_{row}$ |
|--------------------|-------------|-------------------|----------------|
| <i>video</i>       | 4000        | 3500              | 7500           |
| $\overline{video}$ | 2000        | 500               | 2500           |
| $\Sigma_{col}$     | 6000        | 4000              | 10,000         |

### 3. Which Patterns Are Interesting? Pattern Evaluation Methods

#### 3.2. From Association Analysis to Correlation Analysis

- The second correlation measure that we study is the  $\chi^2$  measure.
- To compute the  $\chi^2$  value, we take the squared difference between the observed and expected value for a slot ( $A$  and  $B$  pair) in the contingency table, divided by the expected value. This amount is summed for all slots of the contingency table.

**Table 4.7** Table 4.6 contingency table, now with the expected values.

|                    | <i>game</i> | $\overline{game}$ | $\Sigma_{row}$ |
|--------------------|-------------|-------------------|----------------|
| <i>video</i>       | 4000 (4500) | 3500 (3000)       | 7500           |
| $\overline{video}$ | 2000 (1500) | 500 (1000)        | 2500           |
| $\Sigma_{col}$     | 6000        | 4000              | 10,000         |

### 3. Which Patterns Are Interesting? Pattern Evaluation Methods

#### 3.2. From Association Analysis to Correlation Analysis

- **Example 4.9. Correlation analysis using  $\chi^2$ .**

**Table 4.7** Table 4.6 contingency table, now with the expected values.

|                    | <i>game</i> | $\overline{game}$ | $\Sigma_{row}$ |
|--------------------|-------------|-------------------|----------------|
| <i>video</i>       | 4000 (4500) | 3500 (3000)       | 7500           |
| $\overline{video}$ | 2000 (1500) | 500 (1000)        | 2500           |
| $\Sigma_{col}$     | 6000        | 4000              | 10,000         |

- $$\chi^2 = \sum \frac{(observed - expected)^2}{expected} = \frac{(4000 - 4500)^2}{4500} + \frac{(3500 - 3000)^2}{3000} + \frac{(2000 - 1500)^2}{1500} + \frac{(500 - 1000)^2}{1000} = 555.6$$

- Because the  $\chi^2$  value is greater than 1, and the observed value of the slot (game, video) = 4000, which is less than the expected value of 4500, buying game and buying video are negatively correlated.